Articulatory Feature-Based Phonetic Embeddings for Samskrutam: A Metric and Contrastive Learning Approach

A Project / Dissertation as a Course requirement for

Master of Science in Data Science and Computing

Ritick Raj Gupta

24040208005

SRI SATHYA SAI INSTITUTE OF HIGHER LEARNING

(Deemed to be University)

Department of Mathematics and Computer Science

Nandigiri Campus

March, 2026

DEPARTMENT OF MATHEMATICS & COMPUTER SCIENCE
SRI SATHYA SAI INSTITUTE OF HIGHER LEARNING
(Deemed to be University)

Dept. of Mathematics & Computer Science
Nandigiri Campus

CERTIFICATE

This is to certify that this Project / Dissertation titled **Articulatory Feature-Based Phonetic Embeddings for Samskrutam: A Metric and Contrastive Learning Approach** submitted by Ritick Raj Gupta, 24040208005, Department of Mathematics and Computer Science, Nandigiri Campus is a bonafide record of the original work done under my/our supervision as a Course requirement for the Degree of M.Sc in Data Science and Computing.

…..……..
Dr. V Bhaskaran
Project / Dissertation Supervisor

Countersigned by

Place: Nandigiri

..……..…
Prof. N. Uday Kiran
Date: March 28, 2026 Head of the Department

DECLARATION

The Project / Dissertation titled **Articulatory Feature-Based Phonetic Embeddings for Samskrutam: A Metric and Contrastive Learning Approac** was carried out by me under the supervision of Dr. V Bhaskaran, Department of Mathematics and Computer Science, Nandigiri Campus as a Course requirement for the Degree of M.Sc in Data Science and Computing and has not formed the basis for the award of any degree, diploma or any other such title by this or any other University.

……..

Place : Ritick Raj Gupta

24040208005

Date : II M.Sc

Nandigiri Campus

Acknowledgement

I would like to express my deepest gratitude to Bhagwan Sri Sathya Sai Baba for His divine blessings and guidance. I humbly place this work at His lotus feet.

I sincerely thank Dr. Sampath Lonka for his unwavering support, valuable technical insights, and continuous motivation throughout the entire course of this project. His guidance and mentorship have been instrumental from inception to completion.

I am also grateful to Dr. V. Bhaskaran for his academic support and moral encouragement, which greatly contributed to the successful completion of this work.

I extend my thanks to Svarupa.org for providing the necessary datasets that were essential for carrying out this project.

Contents

1. Abstract 1
2. Introduction 2
3. Literature Review 3
3.1 Phonetic theory (IPA) and articulatory Representation 3
3.2 PanPhon: A resource for mapping IPA segments(phonemes) to Articulatory features vector 3
3.3 PWESUITE 3
3.4 Sequence Modeling using Bidirectional LSTM 3
4. Dataset Description 4
4.1 IPA chart 4
4.2 Rigveda Terms 4
5. Methodology 5
5.1 Articulatory feature vector 5
5.2 PanPhon Articulatory Distance 5
5.3 Triplet Dataset 6
5.4 Model Architecture 6
5.5 Encoder Models developed 7
5.5.1 Triplet Margin Loss model 7
5.5.2 Metric Learning Model 9
5.5.3 Contrastive learning model 12
6. Implementation 15
6.1 Programming Environment 15
6.2 Data Preparation 15
6.3 Model and training configuration 15
7. Results and analysis 16
7.1 Evaluation Metrices 16
7.1.1 Pearson and Spearman Correlation 16
7.1.2 Retrieval Percentile [17] 16
7.1.3 Analogy accuracy [16] 16

7.2 Results from metrices across the models 17
7.3 Analysis 17
8. Conclusion 19
9. Application and Future works 20
9.1 Phonetic Search Engine 20
9.2 Spelling Correction System [21] 20
9.3 Keyphrase extraction [12,13] 21
9.4 Text-to-Speech (TTS) Integration [22] 21
10. References 22

1. Abstract

The present deep learning models that provide embedding vectors for linguistically and grammatically rich languages like Sanskrit give strong representations by handling rich morphology and word order. However, in the case of Sanskrit, along with word order, the phonetic sequence and nuances also matter a lot. This aspect is being overlooked in current trending models like ELMo and Indibert.

This project aims to learn meaningful phonetic representation in form of vector also known as embeddings of Sanskrit words using deep learning techniques. Sanskrit words first converted into a sequence of phonemes, where each phoneme is represented using a 34-dimensional articulatory feature vector. These sequences are then processed using bi-directional LSTM model to capture phonetic nuances and context.

The model generates a fixed-length word embedding using aggregated pooling techniques, including attention-based pooling. The resulting embedding are projected into a lower-dimensional space. Out of three models developed, where models are optimized with Triplet Margin Loss, the Contrastive Learning Info NCE loss ensures that phonetically similar words are embedded closer while dissimilar words are pushed apart, whereas metric learning loss aligns distances in the learned embedding space with linguistically grounded articulatory distances.

Unlike conventional approaches that rely on symbolic similarity or supervised classification, the proposed method directly models continuous phonetic relationships, enabling a geometrically meaningful embedding space. This study highlights the effectiveness of integrating articulatory phonetics with neural representation learning for low-resource and morphologically rich language like Sanskrit.

2. Introduction

Sanskrit is a highly structured and linguistically rich language characterized by its well-defined grammar, extensive morphology, and precise phonetic system that plays a critical role in meaning and pronunciation. Unlike many modern languages, Sanskrit places significant importance not only on syntactic structure and word order but also on phonetic composition, where a small variation in sound can influence the meaning and interpretation. This dual dependency on grammatical and phonetic features makes Sanskrit a challenging yet valuable language for natural language processing tasks.

Recent advancements in deep learning have led to the development of powerful embedding models such as ELMo and IndicBERT, which are capable of capturing contextual and semantic relationships effectively. These models perform well for many languages by leveraging large corpora and contextual learning. However, they are primarily designed to model syntactic and semantic information, although they often overlook the phonetic structure of words. This limitation becomes particularly significant in the context of Sanskrit, where phonetic sequences and articulatory properties play a crucial role.

To address this gap, it is necessary to develop representation learning approaches that explicitly incorporate phonetic information into the embedding process. Phoneme-level modeling, combined with articulatory feature representations, provides a more linguistically grounded framework for capturing sound-based relationships between words. Such representations are especially useful for tasks involving pronunciation, phonetic similarity, and speech-related applications.

In this work, we propose a deep learning-based framework that focuses on learning phonetic-aware embeddings for Sanskrit words. By transforming words into sequences of phonemes and encoding them using articulatory feature vectors, the model captures both phonetic structure and contextual dependencies. The use of bidirectional sequence models further enhances the representation by incorporating information from both forward and backward contexts.

3. Literature Review

3.1 Phonetic theory (IPA) and articulatory Representation

Sanskrit phonology is highly structured and can be effectively described using articulatory features such as place of articulation, manner of articulation, voicing, and aspiration from our mouth. According to the IPA(International Phonetic Alphabet) [26] based Sanskrit phoneme chart , vowels are categorized based on tongue height (close, mid, near-open), tongue position (front, central, back), and lip rounding. Additionally, vowel length (short vs long) plays a phonemic role, distinguishing meanings between otherwise similar words. Consonants in Sanskrit exhibit an even richer articulatory organization. They are systematically arranged based on their place of articulation, including bilabial, dental, retroflex, palatal, and velar categories. Furthermore, consonants are distinguished by voicing (voiced vs voiceless) and aspiration (aspirated vs unaspirated). Additionally, certain sounds such as 'ऋ' (r̥ ɔ) and 'ऌ' (l̩ɔ/l̩ɔ) involve a combination of consonantal and vocalic features, reflecting the intricate nature of Sanskrit phoneme formation.

3.2 PanPhon: A resource for mapping IPA segments(phonemes) to Articulatory features vector

PanPhon is a technique that maps phonetic segments, represented in the International Phonetic Alphabet (IPA), to a set of ternary articulatory representation. These features encode phonological properties such as voicing, nasality, continuance, place of articulation, and manner of articulation. Each phoneme is thus represented as a fixed-length vector, enabling direct use in machine learning models. Each phoneme is represented as vector of ternary values {-1, 0, +1} [2].

3.3 PWESUITE

This sets some techniques to develop models that uses articulatory features to build phonetically informed word embeddings. This also addresses the inconsistent evaluation of existing phonetic word embedding methods, where the evaluation tasks include retrieval, analogy and correlation metrics. The key strategies highlighted in PWESuite [1] is the use of sequence-based encoders, such as recurrent neural networks and convolutional models, to process phoneme sequences.

3.4 Sequence Modeling using Bidirectional LSTM

Modeling phoneme sequence requires architecture that captures temporal and contextual relationships within a sequence. RNN's have been widely used for sequence modeling tasks due to their ability to process variable-length inputs. While BiLSTM [10] network effectively captures both past and future context.

In this work, a BiLSTM-based encoder is employed to process sequence of articulatory features vectors derived form phonemes. Buy leveraging bidirectional context, both local and global phonetic patterns, forming a robust foundation for generating phonetic-aware word embeddings.

4. Dataset Description

4.1 IPA chart

A total of 52 unique phonemes (both vowels and consonants) were found, where each phoneme has their respective articulatory feature representation [26].

1.1. Vowel Chart:

	Front				Central				Back			
	Unrounded		Rounded		Unrounded		Rounded		Unrounded		Rounded	
	Short/Lax (ह्रस्वः)	Long/Tense (दीर्घः)	Short/ Lax	Long/ Tense	Short/ Lax	Long/ Tense	Short/ Lax	Long/ Tense	Short/ Lax	Long/ Tense	Short/ Lax	Long/ Tense
Close	i (इ)	iː (ई)									u (उ)	uː (ऊ)
Close-mid												
Mid	e̞ (ऎ)	e̞ː (ए)									o̞ (ऒ)	o̞ː (ओ)
Near-open					ɐ (अ)	ɐː (आ)					ɒ (अ)	
Near-open Close Diphthong		ɐɪ̯ː (ऐ)										ɐʊ̯ː (औ)

Based on Active Articulator–>		Periferal		Periferal		Laminal		Apical		Laminal		Apical		Apical		Apical		Apical		Periferal		Periferal		Periferal		Peripheral	
Based on Passive Articulator–>		Bilabial		Labiodental		Dental		Alveolar				Retroflex/Cerebral		Palatoalveolar (weakly palatalized)		Alveolopalatal (strongly palatalized)		Palatal		Velar		Uvular/Postvelar		Pharyngeal		Glottal	
Voicing/ Strength –>		Voiceless/ Fortis (ह्रस्व)	Voiced/ Lenis (दीर्घ)	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced	Voiceless	Voiced
Plosive Consonant ('Stop')	Unaspirated	p (प)	b (ब)			t̪ (त)	d̪ (द)					ʈ (ट)	ɖ (ड)					c (च?)	ɟ (ज?)	k (क)	g (ग)						
	Aspirated	pʰ (फ)	bʰ (भ)			t̪ʰ (थ)	d̪ʰ (ध)					ʈʰ (ठ)	ɖʰ (ढ)					cʰ (च?)	ɟʰ (ज?)	kʰ (ख)	gʰ (घ)						
Affricate Consonant (Begins as plosive & ends as fricative)	Unaspirated													t̪ʃ (च?)	d̪ʒ (ज?)	ʈʂ (च)	ɖʂ (ज)										
	Aspirated													t̪ʃʰ (च?)	d̪ʒʰ (ज?)	ʈʂʰ (छ)	ɖʂʰ (झ)										
Nasal		m (म)				n̪ (न)		n̪ (न)				ɳ (ण)		ⁿt̪ʃ (च?)		ⁿʈʂ (च)		ɲ (ञ?)		ŋ (ङ)							
Trill										r̪ (र)																	
Flap/Tap						ɾ̪ (र)				ɾ̪ (र)																	
Lateral Flap/Tap													ɭ (ळ)														
Approximant				ʋ (व)															j (य)								
Lateral Approximant							l̪ (ल)				l̪ (ल)		ɭ (ळ)														
Fricative		ɸ (फ)				s̪ (स)				s̪ (स)		ʂ (ष)		ʃ (श?)		ɕ (श)		ç (श?)		x (ख)						h (ः)	ɦ (ह)
Lateral Fricative																											

Fig[1] from International Phonetic Alphabet (IPA) [26]

4.2 Rigveda Terms

A total of **31,064 unique Sanskrit words** were extracted from a corpus containing **183,422 words** derived from Rigveda texts. This preprocessing step ensured that duplicate entries were removed, resulting in a clean and diverse vocabulary suitable for model training. Each word was then converted into its corresponding phoneme sequence and further represented using **34-dimensional articulatory feature vectors**, forming the base input for the models.

*The above dataset are from svarupa.org

5. Methodology

5.1 Articulatory feature vector

From IPA chart, extracted 51 phonemes and their respective 34 unique articulatory features. While each phoneme can be represented as a 34-dimensional vector of ternary representation {-1, 0, +1}. Where '+1' indicates that the respective articulatory feature is present for that particular, '-1' indicates that the respective articulatory feature is absent and '0' says that the articulatory feature is irrelevant for that phoneme.

For a given Sanskrit word which is a sequence of multiple phonemes, a set of articulatory features for one phoneme and a collection of phonemes forms a word. The articulatory feature for each phoneme is aggregated across features and forms a one vector (of 34-dimensional) representation for one word.

*Hence these forms as base for our further development works.

Word: सार्थम्

स -> [-1.0, -1.0, -1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

ा -> [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

र -> [1.0, -1.0, -1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

् -> [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

थ -> [-1.0, 1.0, -1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

म -> [1.0, -1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

ः -> [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

5.2 PanPhon Articulatory Distance

To calculate the edit distance between any two given words, we took the idea from PanPhon [1,2,3][Mortensen et al.,] and levenshtein distance [19]. Where the insertion and deletion cost is set to 0.5, while the substitution cost between the segments is calculated as,

$$d_{\text{seg}}(a_i, b_j) = \frac{1}{34}\sum_{k=1}^{34}\left|f_k(a_i) - f_k(b_j)\right|$$

This PanPhon Articulatory distance forms as a ground for the next step execution for finding the closest word from a given anchor word.

5.3 Triplet Dataset

This is a dataset generated dataset from the Rigveda terms used for the training and testing of triplet margin model and contrastive model. This Dataset have three elements the anchor word , positive word and the negative word. The positive word is nearest word for the anchor word, while negative word is the farthest word to anchor, when calculated the distance (PanPhon Articulatory distance) between them. This triplet helps us to train the encoder models.

5.4 Model Architecture

This is the core implementation of the work.

The major contribution about the architecture includes three major works:

a. BiLSTM
b. Attention
c. Projection layer

Each Sanskrit word is first converted into a sequence of phonemes also (known as segmentation). Each phoneme is then represented using 34-dimensional articulatory feature vector, in a sequence. For example ["कात”] to ["क", "आ", "त"], a word woukd look like [P1, P2, …..,Pn]. Where each P1 is a 34-dimensional vector. This conveys that the model will not see characters rather it will see phonetic features (how sounds are produced)

To process this sequence, we use Bidirectional LSTM Encoder [10].

Why BiLSTM? A normal neural network cannot handle the word sequences well. Since, LSTM remembers the important information and forgets the unnecessary parts also handles long sequences without losing context. Whereas the BiLSTM does the sequence reading in both the directions (right to left and left to right) preserving the information from both the ends.

The sequence of phonemes are passed to BiLSTM encoder, where we will not get one vector rather we get hidden vectors at each time step as h1, h2, h3, ….. ,hT. This ‘T’ is the length of the phoneme sequence, where encoder takes each phoneme at a time ‘i’ and produces hidden vector ‘i+1’. Number of hidden dimension is set to 256, hence with bidirectional the output dimension will be 2 X hidden_dim = 512

Why Attention? In a sequence of phonemes of a word, not all phonemes contribute equally to pronunciation. Some sounds are more important while some are less important. This is done by assigning weights (also known as importance score) to each hidden state. This helps to learn which phoneme matter more.

Attention mechanism is useful, as it helps in focusing stressed sounds, distinctive articulations and meaningful phonetic variations from a sequence of phonemes. In the final state, all the hidden states are combined using these weights. Hence the **Word Representation =** $\sum \alpha_i h_i$. Thus, this produces a single vector for a whole word. Where α = softmax(W.hi).

Why Projection? The final produced dense representation for a word will be of high-dimensional. So, we need to pass it through projection layer to convert the final hidden vector into a fixed-size embedding to reduce dimensionality. This makes it suitable for training with loss function

Hence the final output will be a vector that represents how a word sounds.

5.5 Encoder Models developed

In all three encoders, the common input to the model is sequence of phoneme-level articulatory feature vectors [11]. Each phoneme is represented as a 34-dimensional vector, and a word which is represented as a variable-length sequence: $X = [X_1, X_2, X_3, \ldots, X_T]$, $X_i \in \mathbf{R}^{34}$

Since different words have different lengths, padding is applied and the corresponding sequence lengths are maintained for efficient processing.

5.5.1 Triplet Margin Loss model

This encoder takes a sequence of vectors derived as articulatory features for each phoneme in a word. The dataset used for model training is structured as triplets, where each triplet consists of an **anchor**, a **positive**, and a **negative** word. Each word in the triplet is represented as a sequence of $n$vectors corresponding to its phoneme sequence[1,4,11]].

Thus, the dataset is constructed in the form of tensors. Since each word in a triplet can have a variable number of phonemes, the input tensors have different lengths. For example, a triplet can be represented as (anchor_tensor, positive_tensor, negative_tensor) , with respective shapes $(L_a, 34), (L_p, 34), (L_n, 34)$, where L_a, L_p, L_ndenote the phoneme lengths of the anchor, positive, and negative words.

However, LSTM requires inputs of equal length within a batch, while words naturally have different phoneme lengths. This issue is handled using a **collate function**, which defines how the DataLoader combines multiple samples into a batch.

Collate Function

For example, consider a triplet:

(A: कर्म, P: प्रवाह, N: संसार).

Initially:

- कर्म has 3 phonemes
- प्रवाह has 4 phonemes
- संसार has 5 phonemes

Since the maximum phoneme length in this triplet is 5, the shorter sequences are padded with zeros to match this length:

- कर्म → padded to (5, 34)

- प्रवाह → padded to (5, 34)
- संसार → already (5, 34)

Along with the padded tensors, the true lengths of each word are also passed (e.g., [3, 4, 5]).

The LSTM now processes all sequences with the same length. However, using sequence packing (pack_padded_sequence), the model is informed about the actual lengths. This ensures that after 3 time steps the first word is considered complete, after 4 time steps the second word is complete, and the padded zeros are ignored. As a result, the final hidden states are computed only from real phoneme inputs and not from padded values. This masking mechanism effectively eliminates the influence of padding.

Attention Mechanism

The attention mechanism [1,8] helps the model focus on important phonemes in the sequence. Each hidden state $h_t$is assigned a score using a linear layer:

$$S_t = W \cdot h_t$$

These scores are converted into attention weights using softmax:

$$\alpha_t = \text{softmax}(S_t)$$

The final representation of the word is computed as a weighted sum of hidden states:

$$h = \sum \alpha_t \cdot h_t$$

This allows the model to give more importance to phonemes that contribute more to the phonetic structure of the word.

Projection Layer

The projection layer reduces the dimensionality of the final hidden representation and transforms it into a fixed-size embedding vector [12]. In this model, the output is a **128-dimensional embedding**.

Model Flow

The overall architecture of the model can be summarized as:

Word → Phoneme Vectors → BiLSTM → Attention → Projection → Embedding (z)

Triplet Margin Loss

The Triplet Margin Loss function [1,8] is used in conjunction with the TripletModelEncoder to enforce relative similarity between samples. Each training sample consists of:

- Anchor (a): a reference word
- Positive (p): a phonetically similar word
- Negative (n): a phonetically dissimilar word

The objective is to bring the anchor and positive embeddings closer together while pushing the anchor and negative embeddings farther apart.

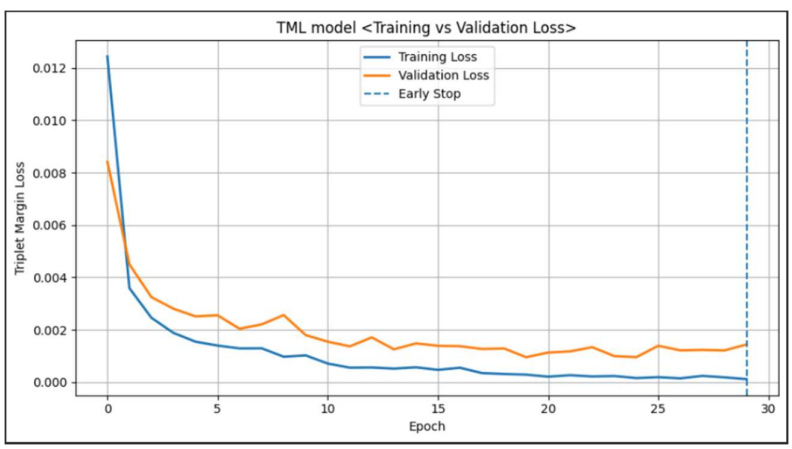
The loss function is defined as:

$$L = \max\left(d(a,p) - d(a,n) + m, 0\right)$$

Where ‘d’ is the PanPhon Articulatory distance and ‘m’ is the margin that enforces separation.

```python
class TripletModelEncoder(nn.Module):
    def __init__(self, input_dim,
                 hidden_dim=256,
                 layers=2,
                 proj_dim=128):
        super().__init__()

        self.lstm = nn.LSTM(
            input_dim,
            hidden_dim,
            num_layers=layers,
            batch_first=True,
            bidirectional=True
        )

        self.attn = nn.Linear(hidden_dim * 2, 1)

        self.proj = nn.Linear(hidden_dim * 2, proj_dim)

    def forward(self, x, lengths):

        packed = nn.utils.rnn.pack_padded_sequence(
            x, lengths.cpu(), batch_first=True, enforce_sorted=False
        )

        out, _ = self.lstm(packed)
        out, _ = nn.utils.rnn.pad_packed_sequence(out, batch_first=True)

        # mask for padded positions
        mask = torch.arange(out.size(1)).to(lengths.device)[None, :] < lengths[:, None]

        scores = self.attn(out).squeeze(-1)   # [batch, seq_len]
        scores = scores.masked_fill(~mask, -1e9)

        weights = torch.softmax(scores, dim=1)  # [batch, seq_len]
        pooled = torch.sum(out * weights.unsqueeze(-1), dim=1)

        return self.proj(pooled)
```

(2)The encoder code for Triplet margin model



(3) Graph showing the training of the Triplet margin model

5.5.2 Metric Learning Model

This encoder [1,5,7] also takes a sequence of vectors where each vector represents the articulatory features of a phoneme. Each word is converted into a sequence of 34-dimensional vectors based on its phoneme structure.

For example:

कर्म → [क, र, म]

Each phoneme → 34-dimensional vector
Final representation → a matrix of size **(3 × 34)**

Since different words have different numbers of phonemes, each word is represented as a variable-length sequence of vectors.

Siamee BiLSTM Encoder

This model follows a Siamese architecture [10,11], where the same encoder is used to process two different input words.

- Word A → Encoder → Embedding A
- Word B → Encoder → Embedding B

Both words pass through the same BiLSTM encoder, meaning:

- The weights are shared
- The model learns a consistent representation for all inputs

The encoder is based on a Bidirectional Long Short-Term Memory, which processes the phoneme sequence in both forward and backward directions. This helps capture phonetic context from both sides of the sequence.

Handling Variable-Length Sequences

Since words have different phoneme lengths, padding is applied to make all sequence in a batch to convert them into same length. Along with padding, the true lengths are also provided. Using sequence packing (pack_padded_sequence), the model:

- Processes only the real phoneme inputs
- Ignores the padded zeros

This ensures that the final representation is the actual phoneme but not the padded zero.

Sequence Representation [11]

After passing through the BiLSTM, the model uses the final hidden states from the last LSTM layer:

- Forward final state → Captures information from beginning to end
- Backward final state → Captures information from end to beginning

These two vectors are concatenated to form a single representation: $h=[h_{forward};h_{backward}]$, the combined vector represents the entire word.

Projection Layer

The Concatenated hidden state is passed through a linear projection layer to obtain a fixed-size embedding. From the input dimension 2 x hidden_dim the vector dimension reduces to 128-dimensionl vector [12].

Metric Learning Loss

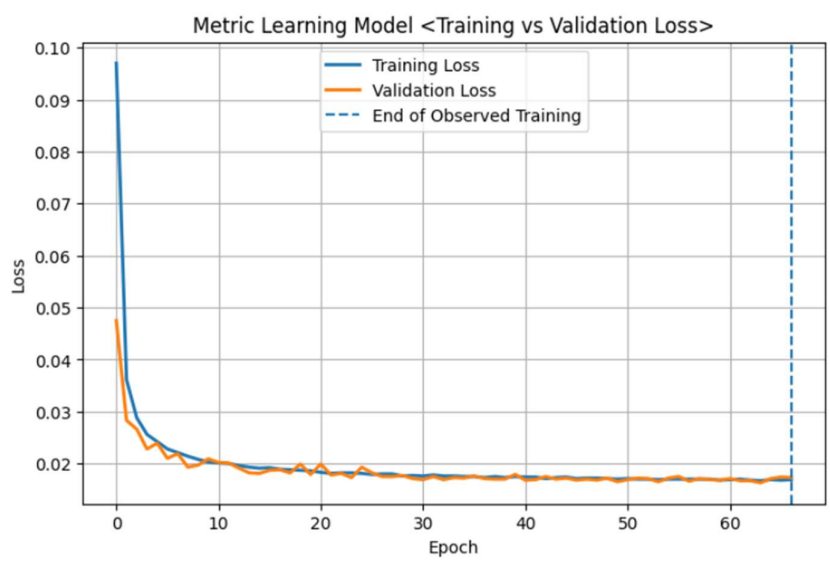
The model is trained using a regression-based loss that aligns embedding distance with articulatory distance.

$$L = \|d_{embed}(x_i,x_j) - A(x_i,x_j)\|^2$$

Where, $d_{embed}(x_i,x_j)$ is the Euclidean distance between the embedding vector of x_i and x_j, and $A(x_i,x_j)$ is the articulatory distance between word x_i and word x_j (the ground truth).

Model flow

Word A → Encoder → emb(A)
Word B → Encoder → emb(B)
} Euclidean Distance → Compare with Articulatory Distance → Loss



(4) Graph showing the training of the Metric Learning model

```python
class MetricEncoder(nn.Module):
    def __init__(self, input_dim=34, hidden_dim=256, layers=2, emb_dim=128):
        super().__init__()

        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=hidden_dim,
            num_layers=layers,
            batch_first=True,
            bidirectional=True
        )

        # because BiLSTM → hidden_dim * 2
        self.proj = nn.Linear(hidden_dim * 2, emb_dim)

    def forward(self, x, lengths):

        packed = nn.utils.rnn.pack_padded_sequence(
            x, lengths.cpu(), batch_first=True, enforce_sorted=False
        )

        _, (h_n, _) = self.lstm(packed)

        # h_n shape:
        # (layers * directions, batch, hidden_dim)

        # last layer forward
        forward_h = h_n[-2]

        # last layer backward
        backward_h = h_n[-1]

        h = torch.cat((forward_h, backward_h), dim=1)

        emb = self.proj(h)

        return emb
```

(5) Encoder code for Metric Learning model

5.5.3 Contrastive learning model

This model learns phonetic embeddings using a contrastive learning approach [1,8] . Each Sanskrit word is first converted into a sequence of phonemes, and each phoneme is represented using a 34-dimensional articulatory feature vector.

Input Representation

For example:

कर्म → [क, र, म]

Each phoneme → 34-dimensional vector

Final representation → matrix of size **(3 × 34)**

Thus, each word is represented as a sequence of phoneme feature vectors. Since different words have different lengths, sequences may vary in size.

Model Architecture

The model follows the following pipeline, Given a word, does **phoneme segmentation** for the phoneme sequence in the word and converts it into 34-dimensional articulatory feature matrix. This is being passed on to **BiLSTM Encoder** [5,7]**,** for which attention pooling is done so that all the words are in same order. Thus with the help of projection head, dimension of the final hidden state vector is reduced to 128-dimensional with normalization. Then this is optimized with InfoNCE Loss.

BiLSTM Encoder [10]**,** process the phoneme sequence in both direction, and for each phoneme the model produces a hidden representation.

Attention Pooling mechanism helps the model to focus on important phonemes. Such that $S_t = W \cdot h_t$, this converts to weights $\alpha_t = \text{softmax}(S_t)$, and the final vector is the weighted sum, given as $h = \sum \alpha_t \cdot h_t$

Projection Head

The pooled vector is passe through a projection head as follows, Linear → ReLU → Linear. This helps the model to learn transformations of feature in better manner. Hence, the final out will be a 128-dimensional embedding vector.

Normalization, L2 normalization (z=‖z‖ / z) is done to ensure that all embedding lie on the same scale so that the similarity comparison becomes stable.

Contrastive Learning setup

The model is trained using pairs of samples with positive pairs (phonetically similar words) and negative samples (phonetically different words). So the final goal is to bring positive pairs closer and push negative samples farther away.

InfoNCE Loss function

The model uses the InfoNCE loss [8] to learn embedding.

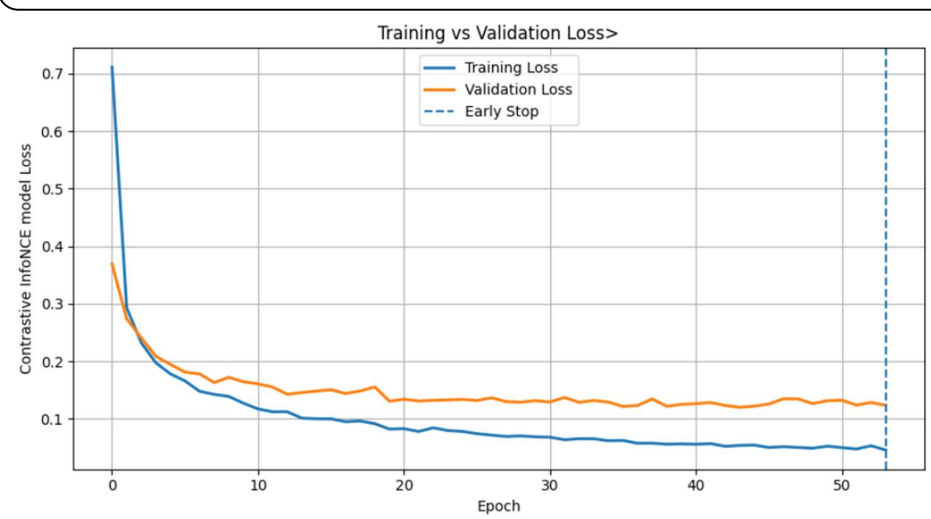
$$L = -\log \frac{\exp\left(\text{sim}(z_i, z_j)/\tau\right)}{\sum_{k=1}^{N} \exp\left(\text{sim}(z_i, z_k)/\tau\right)}$$

Where, z_i, z_j→ embeddings of a positive pair, z_k→ embeddings of other samples (negatives), $\text{sim}(\cdot)$ → similarity (PanPhon Articulatory distance), τ → temperature parameter (controls sharpness)

Model Flow

Word → Phoneme vectors → BiLSTM Encoder → Attention → Projection → Normalization →

→ Embedding z → Compare with other embeddings → InfoNCE Loss.



training of the model

```python
class ContrastiveModelEncoder(nn.Module):
    def __init__(self, input_dim, hidden_dim=128, proj_dim=128):
        super().__init__()

        self.lstm = nn.LSTM(
            input_dim,
            hidden_dim,
            batch_first=True,
            bidirectional=True
        )

        self.attn = nn.Linear(hidden_dim * 2, 1)

        self.projection = nn.Sequential(
            nn.Linear(hidden_dim * 2, hidden_dim * 2),
            nn.ReLU(),
            nn.Linear(hidden_dim * 2, proj_dim)
        )

    def forward(self, x):

        h, _ = self.lstm(x)  # [B, T, 2H]

        attn_scores = self.attn(h)
        attn_weights = torch.softmax(attn_scores, dim=1)

        pooled = torch.sum(attn_weights * h, dim=1)

        z = self.projection(pooled)
        z = F.normalize(z, p=2, dim=1)

        return z
```

(6)Graph showing the Contrastive Learning

(7) Encoder code for Contrastive learning model

6. Implementation

6.1 Programming Environment

- Language: Python
- Framework: PyTorch
- Development: VS Code / Jupyter Notebook / Colab / Kaggle
- Hardware: CPU / GPU

6.2 Data Preparation

Collection of files in .txt format. Cleaning includes remove non-Devanagari characters and normalizing texts.

Phoneme segmentation, to separate phonemes and restoring the sequence

Handling halant issues, when two different phones comes together.

6.3 Model and training configuration

LSTM has 256 hidden dimension with bidirectional , this doubles the dimension size.

Optimizer: Adam

Learning rate: 1e-3

Temperature : 0.1

Margin : 0.4

7. Results and analysis

7.1 Evaluation Metrices

This makes

7.1.1 Pearson and Spearman Correlation

This starts with the measuring of linear relationship between articulatory similarity and Embedding similarity.

For Pearson Correlation [14] ,

- **X**$_{ij}$: Articulatory distance between two words (W_i, W_j)
- **Y**$_{ij}$: Euclidean similarity between the embedding vectors of W_i and W_j.
- Spearman correlation $r = \frac{\sum((xi-\bar{x})(yi-\bar{y})}{\sqrt{\sum(xi-\bar{x})2}\ \sqrt{\sum(yi-\bar{y})2}}$

For Spearman Correlation [15] ,

- **R**x : Sorted order, of the articulatory distance from a selected word to a few set of words.
- **R**y : similarly sorted with Euclidean distance with respect to the embeddings.
- Spearman correlation ρ = corr(**R**x, **R**y)

7.1.2 Retrieval Percentile [17]

Take a word, use articulatory distance to find the n neighbors. And compute the Euclidean similarity with embeddings of these neighbor words and rank them with respect to the articulatory distance.

- Eg. query word = राम => **Rank = 2 , N= len(vocab)**

word	articulatory distance	word	similarity	rank
काम	0.10	नाम	0.92	1
नाम	0.12	काम	0.88	2
गाम	0.30	फल	0.40	3
फल	0.85	गाम	0.39	4
जल	0.90	जल	0.12	5

Percentile = $\frac{N-r}{N}$

7.1.3 Analogy accuracy [16]

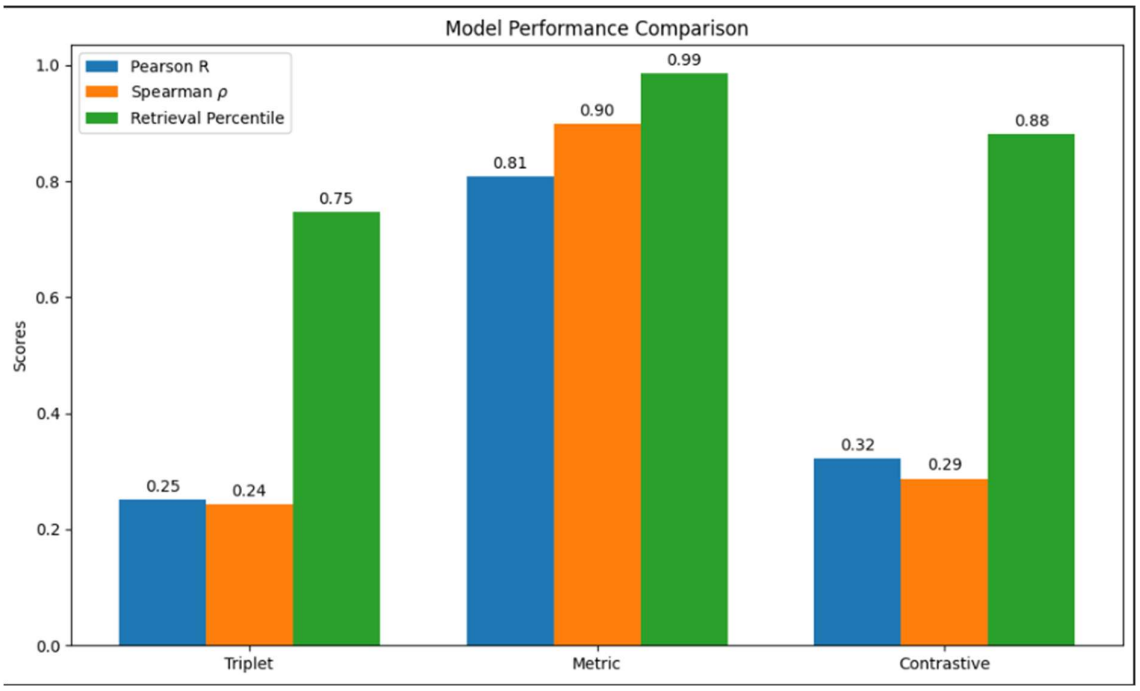
Analogy accuracy evaluates whether the **embedding space captures phonetic relationships**.It tests the classic analogy pattern: a:b::c:d

Where word ‘a’ and word ‘b’ are close, similarly word ‘c’ and word ‘d’ are similar.

First, we compute, Vpred = Vb−Va+Vc . From the calculated arithmetic operation on Vb, Vc and Va we get a vector Vpred. Next, we compute the similarity scores between all the words in corpus with the Vpred. W* = max Sim(Vpred, Vw), is the word that is close to Vpred from vocabulary. If w == d, then analogy is correct.

$$\text{Final Accuracy} = \frac{\text{Number of correct analogies}}{Total\ analogies}$$

7.2 Results from metrices across the models



(8)Graph comparing all the three model form evaluation

7.3 Analysis

The performance of the models was evaluated using multiple metrics to capture different aspects of embedding quality:

- **Pearson Correlation**: Measures linear correlation between embedding distance and articulatory distance
- **Spearman Correlation**: Measures rank-based correlation (monotonic relationship)
- **Retrieval Score**: Measures how well similar words are retrieved
- **Analogy Score**: Measures relational understanding between word pairs

Key Observations

1. Metric Learning Captures True Phonetic Structure

The high correlation values clearly show that:

- Embedding distances closely match articulatory distances
- The model preserves phonetic relationships accurately

2. Retrieval Performance is Strong Across Models

- All models achieve good retrieval scores

- Metric model performs best

Among the three approaches, the **Metric Learning model performs the best**, as it directly aligns embedding distances with articulatory distances. This results in a structured embedding space that accurately reflects phonetic relationships.

The Contrastive model provides a good balance between performance and robustness, while the Triplet model is less effective in capturing detailed phonetic structure.

Example, for the word "पाणिनीय", the embeddings generated from all the three models.

Model	**Sample Embedding (first 6 values)**
Triplet	[-0.1251, 0.0030, -0.0426, 0.0535, -0.1081, 0.1331]
Metric	[0.0481, -0.0072, 0.3289, -0.0402, -0.0095, 0.0036]
Contrastive	[0.1170, 0.1290, -0.0327, -0.0904, -0.0183, -0.0614]

Each model generates a 128-dimensional embedding vector. A small portion of the embeddings is shown for illustration.

8. Conclusion

This project focused on learning meaningful phonetic embeddings for Sanskrit words by integrating articulatory phonetic features with deep learning models. Unlike traditional approaches that rely mainly on textual or semantic similarity, this work emphasizes the importance of **phonetic structure**, which is highly significant in Sanskrit.

In this work, Sanskrit words were first converted into phoneme sequences and then represented using **34-dimensional articulatory feature vectors**. These sequences were processed using a Bidirectional Long Short-Term Memory encoder to capture contextual phonetic information. Three different learning approaches were explored: Triplet Loss, Metric Learning, and Contrastive Learning (InfoNCE).

The experimental results show that the **Metric Learning model performs the best**, achieving high correlation with articulatory distances and excellent retrieval performance. This indicates that the model is able to preserve phonetic relationships in a geometrically meaningful way. The Contrastive model also performs well, especially in retrieval tasks, while the Triplet model shows comparatively lower performance due to its focus on relative similarity rather than exact distance modeling.

Overall, the study demonstrates that incorporating articulatory phonetic features into neural models can significantly improve the quality of embeddings for a morphologically rich and low-resource language like Sanskrit. The learned embeddings are effective for phonetic similarity tasks and provide a strong foundation for further research.

9. Application and Future works

9.1 Phonetic Search Engine

The current system can be extended into a phonetic search engine for Sanskrit texts. In such a system, a user can input a word, and the model retrieves words that are **phonetically similar** [20], even if they are not exactly the same in spelling.

This is particularly useful in Sanskrit because:

- The same word can appear in different forms due to inflections.
- Users may not know the exact spelling

Using embeddings, the system can:

- Convert the query word into an embedding
- Compare it with embeddings of words in a database
- Retrieve the closest matches based on similarity

This enables more flexible and intelligent search compared to exact string matching.

9.2 Spelling Correction System [21]

The learned embeddings can be used to build a phonetic-aware spelling correction system. Unlike traditional spell checkers that rely only on character matching, this system would consider **phonetic similarity**.

For example, If a user types a slightly incorrect word, the system can still find the correct word if it sounds similar

The process would involve:

- Converting the input word into an embedding.
- Finding nearest neighbors in the embedding space.
- Suggesting the closest correct words.

This is especially useful for:

- Handling typing errors.
- Dealing with variations in transliteration.
- Correcting mistakes in complex Sanskrit words.

The proposed phonetic embedding model can be extended to develop a **post-processing correction system for Automatic Speech Recognition (ASR)** [21,22] outputs in Sanskrit.

In ASR systems, especially for Sanskrit, transcription errors are common due to:

- Similar sounding phonemes
- Complex pronunciation rules

- Limited training data
  Instead of relying only on textual correction, the proposed approach uses **phonetic similarity** to correct ASR outputs.

9.3 Keyphrase extraction [12,13]

Phonetic embeddings can also be used for identifying important words or phrases in Sanskrit texts, especially in Vedic literature.

Using embedding similarity and clustering:

- Words with similar phonetic patterns can be grouped together.
- Important or frequently occurring phonetic patterns can be identified.

This can help in:

- Extracting key concepts from texts.
- Summarizing large Sanskrit documents.
- Analysing poetic or chanting structures.

Such an approach is useful where traditional keyword extraction methods may fail due to complex morphology.

9.4 Text-to-Speech (TTS) Integration [22]

This is one of the most important applications of the proposed work. Phonetic embeddings can be integrated into Text-to-Speech systems to improve pronunciation quality. Since the embeddings capture fine phonetic details, they can help in generating more accurate and natural-sounding speech.

In Sanskrit, this is particularly important because:

- Pronunciation directly affects meaning.
- Chanting requires precise phonetic control.

The embeddings can help:

- Model subtle phonetic variations.
- Improve pronunciation consistency.
- Handle rare or unseen words.

This can lead to better TTS systems for:

- Sanskrit chanting.
- Educational audio tools.
- Digital preservation of Vedic recitations.

10. References

[1] M. Zouhar *et al.*, "PWESUITE: Phonetic Word Embeddings and Tasks They Facilitate," *Proc. LREC-COLING*, 2024. [Online]. Available: https://aclanthology.org/2024.lrec-main.1047/

[2] D. R. Mortensen *et al.*, "PanPhon: Mapping IPA to Articulatory Features," *Proc. COLING*, 2016. [Online]. Available: https://aclanthology.org/C16-1328/

[3] A. Parrish, "Phonetic Similarity Embeddings," *Proc. AIIDE*, 2017. [Online]. Available: https://aaai.org/ocs/index.php/AIIDE/AIIDE17/paper/view/15838

[4] S. Sharma *et al.*, "Title not specified," 2021. [Online]. Available: https://arxiv.org/abs/2109.14796

[5] A. Bellet, A. Habrard, and M. Sebban, "Metric Learning," *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 2015. [Online]. Available: https://ieeexplore.ieee.org/document/7047350

[6] X. Chen *et al.*, "Phonetic and Semantic Embeddings," 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8462666

[7] M. Kaya and H. Ş. Bilge, "Deep Metric Learning: A Survey," *Symmetry*, vol. 11, no. 9, p. 1066, 2019. [Online]. Available: https://www.mdpi.com/2073-8994/11/9/1066

[8] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," 2018. [Online]. Available: https://arxiv.org/abs/1807.03748

[9] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," 2015. [Online]. Available: https://arxiv.org/abs/1503.03832

[10] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997. [Online]. Available: https://www.bioinf.jku.at/publications/older/2604.pdf

[11] A. Graves, "Generating Sequences with Recurrent Neural Networks," 2013. [Online]. Available: https://arxiv.org/abs/1308.0850

[12] T. Mikolov *et al.*, "Efficient Estimation of Word Representations in Vector Space," 2013. [Online]. Available: https://arxiv.org/abs/1301.3781

[13] J. Pennington, R. Socher, and C. Manning, "GloVe: Global Vectors for Word Representation," *Proc. EMNLP*, 2014. [Online]. Available: https://aclanthology.org/D14-1162

[14] J. L. Rodgers and W. A. Nicewander, "Thirteen Ways to Look at the Correlation Coefficient," *The American Statistician*, vol. 42, no. 1, pp. 59–66, 1988. [Online]. Available: https://www.jstor.org/stable/2685263

[15] C. Spearman, "The Proof and Measurement of Association between Two Things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904. [Online]. Available: https://www.jstor.org/stable/1412159

[16] T. Mikolov *et al.*, “Linguistic Regularities in Continuous Space Word Representations,” 2013. [Online]. Available: https://arxiv.org/abs/1301.3781

[17] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/

[18] E. M. Voorhees, “The TREC-8 Question Answering Track Report,” 1999. [Online]. Available: https://trec.nist.gov/pubs/trec8/papers/overview_8.pdf

[19] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[20] N. Chomsky and M. Halle, *The Sound Pattern of English*. MIT Press. [Online]. Available: https://doi.org/10.1515/9783110878769

[21] Y. Zhang *et al.*, “Spelling Correction with Phonetic Information,” *Findings of ACL*, 2021. [Online]. Available: https://aclanthology.org/2021.findings-acl.198/

[22] F. Fang *et al.*, “Robust Automatic Speech Recognition,” 2020. [Online]. Available: https://dl.acm.org/doi/10.1145/3397271.3401320

[23] H. Terdalkar and A. Bhattacharya, “Chandojnanam: A Sanskrit Meter Identification and Utilization System.” [Online]. Available: https://github.com/hrishikesht/chandojnanam

[24] T. Gandhi, “Pāṇiniya Śikṣā’s Phonetic Apparatus and its Applications for Modern Languages,” Department of Sanskrit, University of Delhi, New Delhi, India. [Online]. Available: https://www.academia.edu/

[25] “Phonetics and Phonological Rules in Pāṇiniya Śikṣā.” [Online]. Available: https://www.academia.edu/

[26] “International Phonetic Alphabet (IPA) Chart of Sanskrit Phonemes (Based on the Current Standard Phonology of Sanskrit).” [Online]. Available: https://www.internationalphoneticassociation.org/

[27] S. Kumar, P. Ray, A. Beerukuri, S. Kamoji, M. B. Jagadeeshan, and P. Goyal, “Vedavani: A Benchmark Corpus for ASR on Vedic Sanskrit Poetry.” [Online]. Available: https://arxiv.org/abs/2305.12982